

KCL: KittyCAD Language

Syntax

```
x = value;
fn function(@i, x, y?) { return x }
fn function(x: string, y?: string) {}
x |> f() |> g() Same as g(f(x))
```

Operators

```
n + m  add, string concat.
n - m  subtract
-m     negate
n * m  multiply
n / m  divide
n < m  less than
n <= m less or equal
b == c equal to
b != c not equal
n >= m greater or equal
n > m  greater than
b & c  logical and
b | c  logical or
!b     logical not
```

Conditionals

```
x = if cond { 1 } else { 2 }
```

Units

```
dist = 45cm - 2in, ang = 45deg - 2rad
fn f(intoMm: number(mm)): number(in)
```

Arrays

```
arr = [1, 2, 3], arr = [1..3], arr = [1..<4]
squared = map(arr, fn(x) { return x*x })
sum = reduce(
  myArr,
  initial = 0,
  f = fn(@i, accum) {return i + accum}
)
```

Math

```
sin(10deg), cos(1rad), tan(x)
asin(10deg), acos(1rad), atan(x), atan2(y, x)
polar(angle = 9deg, length = 5)
rem(7, divisor = 4)
assert(x, isEqualTo = 3)
sqrt(4)
abs(-4)
round(4.1), floor(4.1), ceil(4.1)
min([1, 2, 3]), max([1, 2, 3])
pow(5, exp = 2), log(100, base = 5)
log10(100), log2(128), ln(100)
```

Modules

```
import wheelDepth, wheelDiameter from "car.kcl"
export wheelDepth = 2cm
```

Other

```
clone(solid)
```

Profiles

```
profile = sketch(on = XZ) { ... }
profile = sketch(on = startSketchOn(solid, face = END)) { ... }
```

Sketching

```
point(at = [var 1, var 1])
line(start = [var 0, var 0], end = [var 3, var 4])
arc(start = [var 4, var 0], end = [var -4, var 0], center = [var 0, var 0])
circle(start = [var 2, var 0], center = [var 0, var 0])
line(..., construction = true) Guide geometry
```

Constraints

```
coincident([line1.end, line2.start])
horizontal(line1), vertical(line2)
distance([line1.start, line2.end]) == 5
radius(arc1) == 4, diameter(circle1) == 8
```

Transforms

```
translate(profile, x = 1.3, z = 2.2)
scale(profile, y = 0.5, z = 2)
rotate(profile, roll = 5deg, pitch = 5deg, yaw = 5deg)
rotate(profile, axis = Z, angle = 45deg)
```

Booleans

```
subtract2d(profile, tool = holeShapeSketch)
```

Patterns

```
patternLinear2d(profile, instances, distance = 10, axis = X)
patternCircular2d(profile, instances, center = [0, 0])
patternTransform2d(profile, instances, transform = someFn)
mirror2d(unclosedPath, axis = Y)
```

Selections

```
region(point = [x, y], sketch = profile) Area/face
```

Tagging

```
edge = line(start = [var 0, var 0], end = [var 3, var 4])
region(point = [1, 1], sketch = profile).tags.edge
chamfer(length = 1, tags = [a], tag = $newFace)
extrude(length = 1, tagEnd = $newFace)
```

Profiles to Solids

```
extrude(region, length = 4, method = MERGE | NEW)
extrude(region, to = endExtrudeAtThis)
sweep(profile, path = path)
loft([profile1, profile2])
```

Solids

Transforms

```
translate(solid, x = 1.3, z = 2.2)
scale(solid, y = 0.5, z = 2)
rotate(solid, roll = 5deg, pitch = 5deg, yaw = 5deg)
rotate(solid, axis = Z, angle = 45deg)
```

Booleans

```
union([solid, solid2])
subtract(cube, tools = [hole])
intersect([solidA, solidB])
```

Patterns

```
patternLinear3d(instances = 4, distance = 10, axis = X)
patternCircular3d(instances, center = [0, 0, 0])
patternTransform(instances, transform = someFn)
```

Modifiers

```
shell(cube, faces = [taggedFace], thickness = 1)
fillet(solid, radius = 5, tags = [taggedEdge])
chamfer(solid, radius = 5, tags = [taggedEdge])
appearance(solid, color = "#00ff00")
```

Example

```
p = sketch(on = XY) {
  l1 = line(start = [var 0mm, var 0mm], end = [var 10mm, var 0mm])
  l2 = line(start = [var 10mm, var 0mm], end = [var 10mm, var 10mm])
  l3 = line(start = [var 10mm, var 10mm], end = [var 0mm, var 10mm])
  l4 = line(start = [var 0mm, var 10mm], end = [var 0mm, var 0mm])
  coincident([l1.end, l2.start])
  coincident([l2.end, l3.start])
  coincident([l3.end, l4.start])
  coincident([l4.end, l1.start])
  horizontal(l1)
  vertical(l2)
  horizontal(l3)
  vertical(l4)
}
r = region(point = [5mm, 5mm], sketch = p)
s = extrude(r, length = 5cm)
```

Links: [KCL book](#) | [Zoo Design Studio](#) | [Language reference](#) | [Sample models](#) | [GitHub](#) | [Treesitter](#) | [LSP](#)

[Edit me on GitHub!](#)

Full credit to the [OpenSCAD cheat sheet](#) for inspiration and design